



國立高雄科技大學

National Kaohsiung University of Science and Technology

AI AGENT

(人工智慧代理)

Automatic Testing by Antigravity IDE

Instructor: Shih-Huan Tseng

shtseng@nkust.edu.tw

Department of Computer and Communication Engineering

National Kaohsiung University of Science and Technology



Automatic Testing – Browser Subagent

- 瀏覽器整合功能

- Agent 可以直接操作瀏覽器

- 開啟網頁、點擊按鈕、填寫表單、擷取畫面、錄製整個操作過程。

- 實作 Todo List 網頁應用

請幫我建立一個 Todo List 網頁應用，需求如下：

功能需求：

1. 可以新增待辦事項（輸入文字後按 **Enter** 或點擊按鈕）
2. 可以標記事項為完成/未完成（點擊切換）
3. 可以刪除事項
4. 可以篩選顯示：全部、未完成、已完成
5. 使用 `localStorage` 儲存資料，重新整理不會遺失

技術要求：

- 使用純 HTML、CSS、JavaScript（不使用框架）
- CSS 使用現代化設計，支援響應式佈局
- JavaScript 使用 ES6+ 語法
- 程式碼要有適當的註解

請先建立專案結構，然後逐步實作各個功能。

Automatic Testing – Browser Subagent

- 開啟瀏覽器預覽
- 查看程式碼
- 下測試任務prompt

請開啟本地檔案

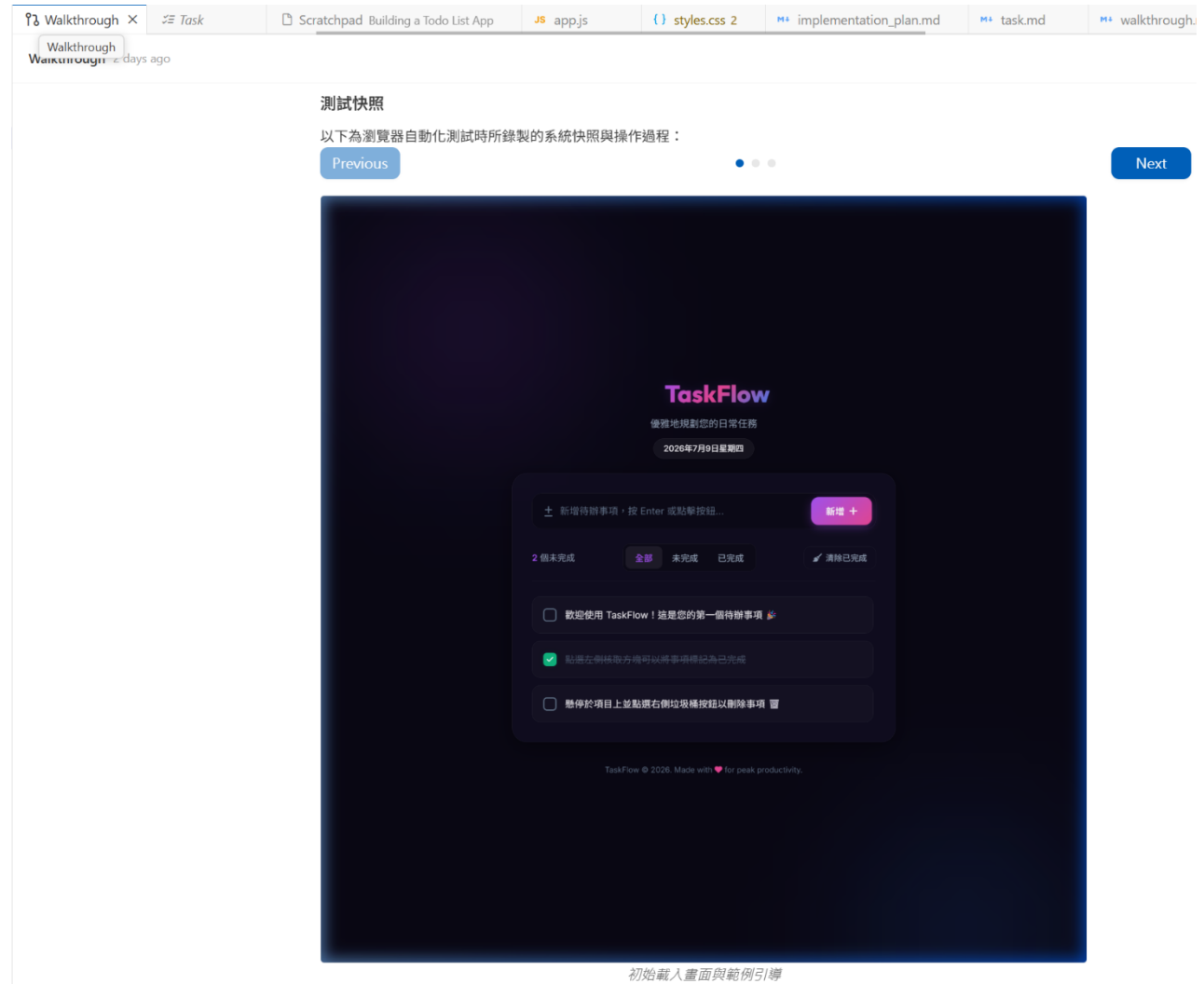
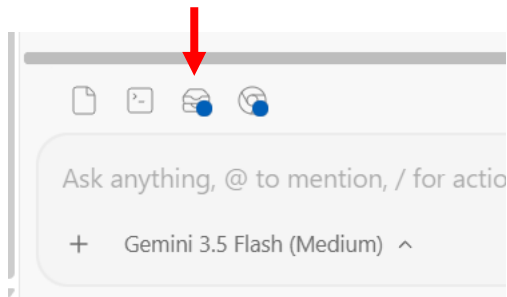
D:\Antigravity Codes\Browser Subagent\todo-app\index.html

並執行以下測試：

1. 檢查頁面標題是否為「我的待辦事項」。
 2. 新增兩個任務：「完成網頁閱讀」與「完成查詢」。
 3. 點擊「完成網頁閱讀」將其標記為完成。
 4. 切換篩選器到「已完成」，確認只看到「完成網頁閱讀」。
 5. 切換回「全部」。
 6. 刪除「完成查詢」。
 7. 重新整理頁面，確認「完成網頁閱讀」依然是完成狀態。
- 請在完成後回報結果與任何發現的問題。

Automatic Testing – User Feedback

- 開啟artifact
- 點擊walkthrough
- 點擊截圖
- 拖曳滑鼠選取範圍
- 下comments



Automatic Testing – User Feedback

p > {} styles.css > ...

```
/ D:\Antigravity Codes\Browser Subagent\todo-app • Contains emphasized items =====  
TaskFlow Core Design System & Stylesheet  
TaskFlow Core Design System & Stylesheet (Light Theme - Fresh Green)  
===== */
```

Accept Alt+⇧ Reject Shift+Alt+⇧

```
/* --- Design Tokens & Variables --- */
```

```
:root {
```

```
--font-heading: 'Outfit', 'Inter', -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;  
--font-body: 'Inter', -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
```

```
/* Theme Colors (Dark Theme) */
```

```
--color-bg: #090714;  
--color-surface-glass: rgba(20, 16, 38, 0.6);  
--color-surface-hover: rgba(255, 255, 255, 0.07);  
--color-border: rgba(255, 255, 255, 0.08);  
--color-border-focus: rgba(168, 85, 247, 0.4);
```

```
/* Theme Colors (Light Theme - Fresh Green/Mint) */
```

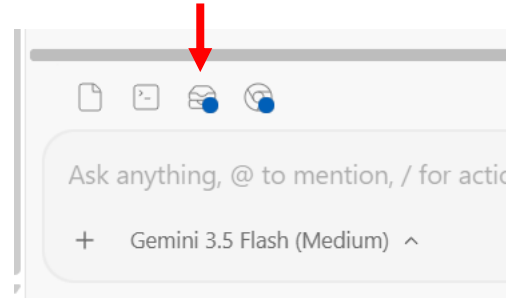
```
--color-bg: #f3f7f5;  
--color-surface-glass: rgba(255, 255, 255, 0.75);  
--color-surface-hover: rgba(16, 185, 129, 0.08);  
--color-border: rgba(16, 185, 129, 0.16);  
--color-border-focus: rgba(16, 185, 129, 0.5);
```

Automatic Testing – User Feedback



Automatic Testing – User Feedback

- 開啟artifact
- 點擊implementation
- 下comments，點擊proceed



Todo List 網頁應用實作計劃

本計劃旨在建立一個擁有現代化美觀設計、響應式佈局且功能完善的 Todo List 網頁應用。我們將在 `todo-app` 資料夾下，使用純 HTML、CSS 與 JavaScript (ES6+) 開發，並使用 `localStorage` 來永久儲存待辦事項。

設計美學與 UX 規劃

為了呈現令人驚艷 (WOW) 的視覺效果，我們將採用以下設計語彙：

1. **暗黑玻璃擬態 (Glassmorphism Dark Mode)**：背景使用漸層流光，搭配半透明的毛玻璃容器面板，極具現代科技感。
2. **漸層與色彩**：使用活力粉紫 (Vibrant Purple/Pink) 到深邃湛藍 (Deep Blue) 的漸層作為主視覺，按鈕與互動元素搭配動態霓虹光暈。
3. **微互動與動畫**：

使用框架而非純HTML、CSS與JavaScript

Cancel

Add Comment

Automatic Testing – User Feedback

- 確認前端框架
- Prompt: 1和3都做，並建立使用說明和說明技術差異

❓ 您希望使用哪一個前端框架來重新建置或重構此 Todo List 待辦清單網頁應用？

- 1 (Recommended) React (使用 Vite 建立)
- 2 Vue (使用 Vite 建立)
- 3 Next.js (React 框架，適合 SSR/路由)
- 4 Other (write your answer)

Skip [Submit ↵](#)

Automatic Testing – User Feedback

- 查看Implementation Plan
- 點擊proceed

React & Next.js Todo List 實作與技術對比計劃



根據使用者的反饋，我們將原先的純 HTML/CSS/JS 專案升級，同時實作 **React (Vite)** 與 **Next.js** 兩個版本的待辦清單網頁應用，並建立詳盡的使用說明與技術差異分析文件。

項目結構規劃

我們將在 `d:\Antigravity Codes\Browser Subagent\` 目錄下建立以下結構：

- `[NEW] todo-react-vite/`：React + Vite 待辦清單應用。
 - `[NEW] todo-nextjs/`：Next.js (App Router) 待辦清單應用。
 - `[NEW] framework_comparison.md`：比較 React (Vite) 與 Next.js 的技術特點、渲染機制、效能表現及啟動運行說明。
-

Automatic Testing – User Feedback

預計修改/新建檔案與工具執行步驟

第一階段：命令參數研究與專案初始化

我們將依照開發指南，首先執行 `--help` 命令了解各框架初始化工具的參數，然後以非互動模式建立專案：

1. 查詢 `create-vite` 的說明選項：`npx -y create-vite@latest --help`
2. 查詢 `create-next-app` 的說明選項：`npx -y create-next-app@latest --help`
3. 建立並初始化 React 專案：
 - 建立資料夾 `todo-react-vite`，並在其中執行：`npx -y create-vite@latest ./ --template react`
4. 建立並初始化 Next.js 專案：
 - 建立資料夾 `todo-nextjs`，並在其中以非互動式參數執行：`npx -y create-next-app@latest ./ --ts=false --js=true --eslint=true --tailwind=false --src-dir=false --app=true --import-alias="@/*"` (使用 Javascript 版本)

第二階段：React (Vite) 實作

- **CSS 整合**：沿用我們先前設計的「淺色翠綠毛玻璃風格」樣式系統（寫入 `src/index.css`）。
- **元件拆分**：
 - `src/App.jsx`：主 App 元件，負責處理 `todos` 狀態邏輯、與 `localStorage` 互動。
 - `src/components/ToDoForm.jsx`：新增待辦事項輸入框。
 - `src/components/ToDoFilter.jsx`：篩選按鈕與清除按鈕列。
 - `src/components/ToDoList.jsx` 與 `ToDoItem.jsx`：渲染事項列表與單個事項，處理勾選與刪除。
- **動畫處理**：在 React 中透過狀態觸發刪除動畫（使用 `onAnimationEnd` 或 CSS transition）。

Automatic Testing – User Feedback

第三階段：Next.js 實作

- **專案結構**：使用 App Router 結構。
 - **CSS 整合**：將淺色翠綠毛玻璃樣式寫入 `app/globals.css`。
 - **元件與客戶端邏輯**：
 - 在 Next.js 中，由於 `localStorage` 僅在客戶端可用，我們將在元件頂部宣告 `"use client"`，並在 `useEffect` 中安全地初始化狀態，避免 SSR (伺服器端渲染) 的 Hydration 不一致問題。
 - 實作與 React 類似的元件拆分結構。
-

第四階段：技術差異與啟動說明文件

- 建立 `framework_comparison.md`，包含：
 1. **技術比較表格**：從架構、路由、渲染方式 (CSR vs SSR/RSC)、打包體積、適用場景等維度進行分析。
 2. **啟動指引**：如何安裝依賴 (`npm install`)、如何啟動開發伺服器 (`npm run dev`) 等。
-

Browser Subagent – 練習

- 爬蟲

- 使用Browser Subagent，瀏覽URL 做重點整理，存成md檔。
 - [Adaptive-RAG](#)
 - [GraphRAG](#)
- 配合skill 將md檔轉成pdf。

- 查詢

- 使用Browser Subagent，瀏覽<https://kpp.tbkc.gov.tw/> 輸入車號xxx-xxxx與驗證碼，確定送出。將結果畫面截圖下來。

Automatic Testing – Multi Agents

- Prompt

我想建立一個「個人記帳應用」，同時使用三個 Agent 分別處理：

Agent 角色核心職責具體負責內容

1. Agent A前端開發負責 HTML / CSS / JavaScript 介面設計與互動。
2. Agent B資料管理負責 localStorage 資料結構設計與資料存取邏輯。
3. Agent C整合測試負責使用 Browser Subagent 進行端到端的自動化功能測試。
4. 協作與工作區規範

- 產生implementation.md

Automatic Testing – Multi Agents

- Antigravity 2.0 : /teamwork-preview
 - 專為大型、複雜專案設計的多 Agent 協同工作系統 (Multi-Agent System) 。
 - 讓多個專屬子代理程式 (包含 Sentinel、Orchestrator、Worker、Reviewer 等) 為您進行大型任務的程式碼編寫與測試。
- 什麼情境使用?

Automatic Testing – Multi Agents

- 階段一：共同規劃任務規格書
 - 明確專案概念 (Step 1)：確認您想建造什麼、用途為何、目標受眾是誰。
 - 釐清模糊地帶 (Step 2)：針對 scope、技術限制等不確定點提供選項供您選擇。
 - 決定誠信模式 (Step 3)：設定 Agent 是否能參考外部 open-source、使用預建庫等。
 - 起草需求 (Step 4)：定義「做什麼 (What)」，但不限制「怎麼做 (How)」，保留給 Agent 團隊設計空間。
 - 設計驗證機制 (Step 5)：規劃客觀的測試方式（如測試腳本），強迫 Agent 進行實質的偵錯與驗證。

Automatic Testing – Multi Agents

- 階段一：共同規劃任務規格書
 - 設定驗收標準 (Step 6)：將驗證結果轉化為明確的 checkable 項目。
 - 基礎設施限制 (Step 7)：規範網路存取、雲端儲存等權限。
 - 選擇工作目錄 (Step 8)：決定專案檔案要存放在電腦的哪個路徑。
 - 組裝與確認 (Step 9)：由您最後審查規格書，確認無誤後授權啟動。

Automatic Testing – Multi Agents

- 階段二：委派與背景執行 (Delegation)
 - 輸入「開始執行」或「確認啟動」後，這份規格書派發給 teamwork_preview 的後台 Agent 團隊。
 - AI 專家團隊（研究、開發、測試等角色）會在背景協同工作，透過不斷地「編寫 → 測試 → 偵錯」循環來優化成果。
 - 執行期間可以繼續進行其他工作，完成後系統會自動通知您，並提供完整的 walkthrough 成果報告。

Automatic Testing – Multi Agents

- 底層框架會自動管理工作區的分配與隔離
 - 自動為前端和資料管理建立獨立的 Git 分支工作區，並在完成後進行合併)
- 獨立的 Workspace（工作區）
 - 防止程式碼衝突與互相覆寫 (Race Condition)
 - 如果多個 Agent 同時在「同一個資料夾」下工作，當 **Agent A** 正在修改某個檔案時，**Agent B** 可能也想修改同一個檔案。這會導致兩者的程式碼互相覆寫，或者產生衝突，使得開發進度混亂。
 - 避免 Context（上下文）與日誌污染

Automatic Testing – Multi Agents

- **獨立的 Workspace（工作區）**

- 避免 Context（上下文）與日誌污染

- Agent 在工作時需要頻繁讀取目錄下的檔案、搜尋程式碼、以及查看終端機日誌。
 - 如果所有 Agent 擠在同一個工作區，編譯產生的暫存檔、測試日誌、指令輸出等會混在一起。
 - 導致 Agent 的 Context 被干擾，讀取到不相關的資訊，進而影響決策與程式碼品質。

- **模擬真實團隊的「分支開發 (Branch/Worktree)」模式**

- 後台通常使用 branch（分支）或 share（類似 Git Worktree）模式來建立子工作區：

- **獨立開發：Agent A (前端) 與 Agent B (資料邏輯)** 可以在各自的沙盒中自由地編寫、修改與測試，互不干擾。
 - **合併與測試：**當各自的模組完成並通過初步驗證後，系統會將兩者的成果合併 (Merge) 回主專案，最後才由 **Agent C (整合測試)** 在合併後的工作區進行全面的端到端 (E2E) 測試。

Automatic Testing – Multi Agents

- /teamwork-preview 「個人記帳應用」

?

這個個人記帳應用的主要用途與開發目的為何？

← 1 of 3 →

1

(推薦) 個人日常使用，著重在美觀的網頁介面、流暢的互動與本地儲存 (LocalStorage)

2

行動裝置優先的響應式網頁，方便隨時隨地用手機記帳

3

純概念驗證 (PoC)，快速實現核心的記帳與統計邏輯，畫面陽春即可

4

Other (write your answer)

Skip

Continue

Automatic Testing – Multi Agents

- /teamwork-preview 「個人記帳應用」

?

您希望包含哪些核心功能？(可多選)

← 2 of 3 →

Multi-select

1

基本收支記錄 (新增、刪除、修改、查看交易明細)

2

交易分類管理 (預設分類如餐飲、娛樂，並支援使用者自訂分類)

3

統計圖表 (以圓餅圖/長條圖展示各類別消費比例與每月收支趨勢)

4

預算管理 (設定每月預算上限，並在超支時發出警示)

5

資料匯出與匯入 (支援將記帳資料匯出為 CSV/JSON 檔案，或從檔案匯入)

6

1-4

Skip

Continue ↗

Automatic Testing – Multi Agents

- /teamwork-preview 「個人記帳應用」

❓ 您對技術棧 (Technology Stack) 有什麼特定偏好嗎？ ← 3 of 3 →

- 1 純前端網頁應用 (HTML + Vanilla CSS + JavaScript，資料存於 LocalStorage)
- 2 全端網頁應用 (前端 HTML/JS + 後端 Node.js 或 Python，搭配資料庫存儲)
- 3 讓 AI 團隊根據需求自行選擇最適合的技術棧
- 4 Other (write your answer)

Skip Submit (Enter) Submit ↩

Automatic Testing – Multi Agents

- /teamwork-preview 「個人記帳應用」

❓ 您希望限制 AI 團隊在開發過程中的哪些行為？(可複選，用以評估誠信模式) ← 1 of 2 →

Multi-select

- 1 複製現有開源專案的核心邏輯程式碼 (Copying code from existing open-source projects)
- 2 使用預先建置的函式庫或框架來實現核心功能，例如 Chart.js 用於圖表 (Using pre-built libraries/frameworks)
- 3 運行外部腳本或將部分工作委託給其他工具執行 (Running external scripts/delegation)
- 4 在實作前閱讀測試程式碼來理解預期的行為 (Reading test source code first)
- 5 無限制 — AI 團隊可以使用任何有效且符合規範的方法 (None)
- 6 Other (write your answer)

Skip [Continue ↗](#)

Automatic Testing – Multi Agents

- /teamwork-preview 「個人記帳應用」

?

關於這個記帳應用的功能驗證，您手邊是否有現成的測試資源？

← 2 of 2 →

1

(推薦) 沒有。請 AI 團隊在專案中自動寫一個客觀的自動化測試腳本（例如 Puppeteer / Playwright 網頁測試），用以驗證所有功能。

2

沒有。在開發完成後，請一個獨立的評審 AI (Agent-as-judge) 根據明確的視覺與功能量規表 (Rubric) 來手動/視覺化驗收。

3

有。我手邊已經有現成的測試腳本或參考實作，待會可以提供給 AI 團隊。

4

Other (write your answer)

Skip

Submit

Automatic Testing – Multi Agents

- Prompt_draft.md
 - prompt_draft.md內容怎麼看出有多代理?



Automatic Testing – Multi Agents

